The Strategy and Decorator Patterns

Welcome back to CS 2100!

Prof. Rasika Bhalerao

Strategy Pattern

You are using the Strategy pattern in Homework 8 without knowing it.

Defining distance

Algorithms for "shortest distance" (root to each node, MST) -> must define "distance"

Part of HW8: defining the distance between train stations

- Basic measure of Euclidean distance (used in code)
- Number of minutes it takes to travel: "the station is 15 minutes from here"
- Financial cost of building th track (which is bigger if we have to dig a tunnel underwater or through a mountain)
- `1/n`, where `n` is the number of passengers who regularly travel between the two stations

Strategy pattern

Many definitions of distance, one algorithm that can use any of them

Strategy (Duration) Pattern: allow user to choose / change distance definition at run time

Ex: Strategies for Tic Tac Toe

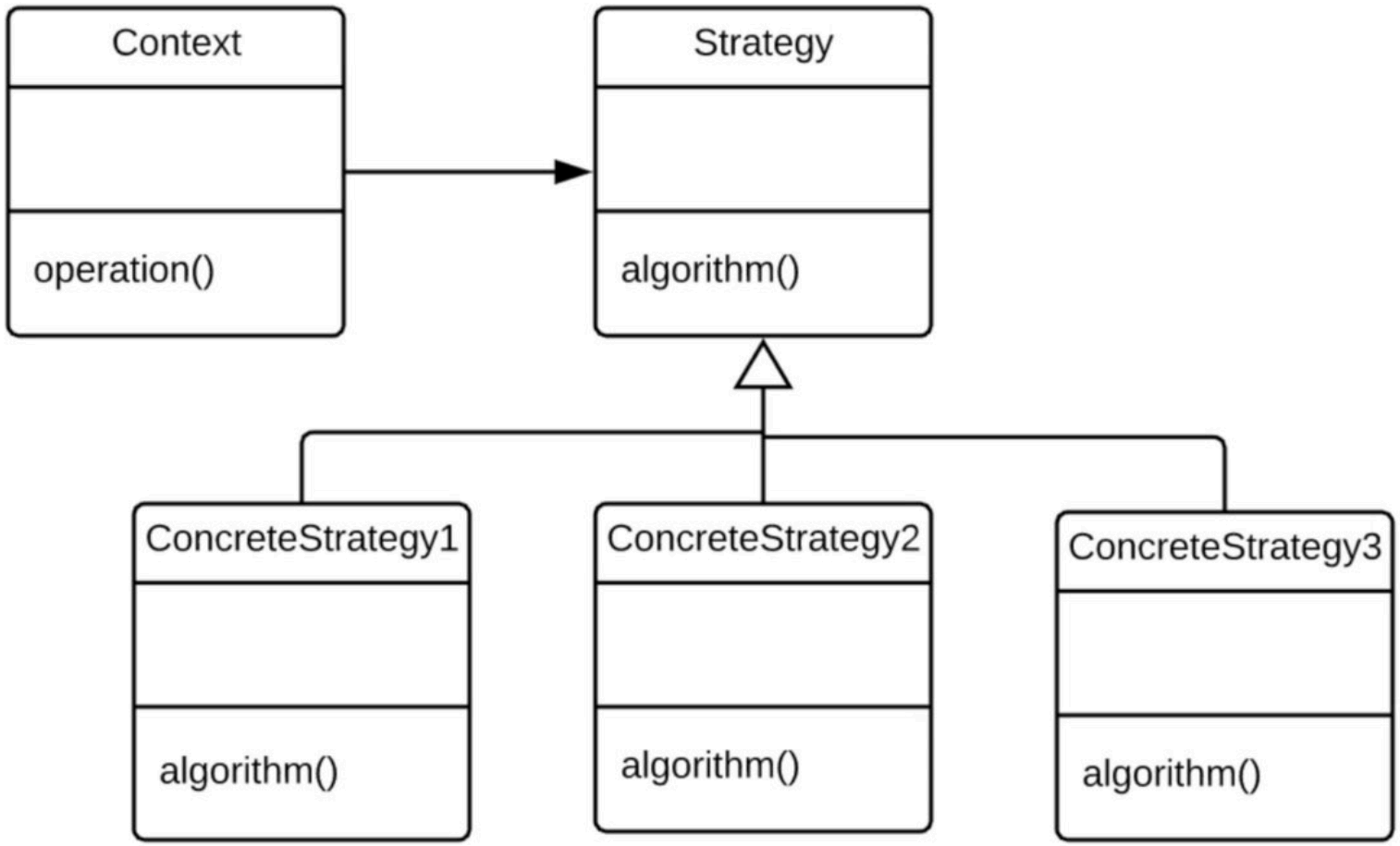
- Place a third piece in a row to win
- Block the opponent if they're about to win
- Place in an open corner
- Place in any open square

Strategy for choosing a place changes based on game state

Ex: maps directions

- Shortest path
- Shortest time
- Least emissions
- Least tolls
- Maximize sightseeing

User chooses the path-finding algorithm at runtime


Context
operation()
Strategy
algorithm()
ConcreteStrategy1
algorithm()
ConcreteStrategy2
algorithm()
ConcreteStrategy3
algorithm()

Poll: How could the Strategy Pattern be used to define multiple definitions of distance in Homework 8 (public transportation MST)?

1. Ignore the distances between stations when building the train track system (assume all stations are equidistant)
2. Take user input to choose between multiple existing implementations of the `distance()` function (which returns the distance between two stations)
3. Take user input to choose between multiple existing implementations of Kruskal's Algorithm (which builds a Minimum Spanning Tree)
4. After the train track system is built, allow passengers to choose where they want to travel using the trains.

Function Decorators

We have already seen some decorators:

- `@property` (to create a property named after the method)
- `*.setter` (to give that property a "setter" method)
- `@classmethod` (for `setUpClass(cls)` and `tearDownClass(cls)` to run once before / after unit testing)

We will find out how these decorators were implemented / how to create our own decorators

Functions are objects that can be stored in mutable variables

We already knew functions are objects:

```
from typing import Callable

def double(num: int) -> int:
    return num * 2

def triple(num: int) -> int:
    return num * 3

functions: list[Callable[[int], int]] = [double, triple]

def apply_functions(num: int) -> list[int]:
    return [func(num) for func in functions]

print(apply_functions(5))  # [10, 15]
```

A function decorator is a way to "mutate" a function to modify what it does.

Function decorators "mutate" functions by wrapping them in other functions

Example: `@time_calls` modifies a given function to print the time it took to run it:

```
@time_calls
def multiply_list(nums: list[int], multiplier: int = 1) -> list[int]:
    return [num * multiplier for num in nums]

print(multiply_list([i for i in range(5)], multiplier = 3))
```

Because we modified `multiply_list()` to do what `@time_calls` tells it to do, this is the printed output:

```
Executed multiply_list with ([0, 1, 2, 3, 4],) and {'multiplier': 3} in 7.152557373046875e-06ms
[0, 3, 6, 9, 12]
```

How we created that `@time_calls` decorator:

```
def time_calls(func: Callable[..., Any]) -> Callable[..., Any]:
    def wrapper(*args: Any, **kwargs: Any) -> Any:
        now = time.time()
        return_value = func(*args, **kwargs)
        print(f'Executed {func.__name__} with {args} and {kwargs} in {time.time() - now}ms')
        return return_value
    return wrapper
```

`time_calls()` takes a function as its argument and returns a function.

The returned function is a modified version of the argument function.

That modification: in addition to running the function as usual, it prints the number of milliseconds that it took to run it.

Poll: What does the `@mystery` decorator do?

```
T = TypeVar('T')

def mystery(func: Callable[[float], T]) -> Callable[[float], T]:
    def wrapper(x: float) -> T:
        if x < 0:
            raise ValueError("Argument must be positive")
        return func(x)
    return wrapper

@mystery
def square_root(x: float) -> Any:
    return x ** 0.5

print(square_root(9))
```

1. Modifies the function to print the number of milliseconds it took to run it
2. Modifies the function such that it raises a `ValueError` if the argument is negative
3. Modifies the function to run it twice
4. Nothing

Caching: a common use of decorators

Caching: temporarily storing values that are recalculated often (so they only need to be calculated once).

```
def fibonacci(n: int) -> int:
    if n < 2:
        return n
    return fibonacci(n-1) + fibonacci(n-2)
```

In order to calculate the 5th Fibonacci number, we need to calculate the 4th and 3rd ones.
And to calculate the 4th number, we need to calculate the 3rd and 2nd ones.
And to calculate the 3rd number, we need to calculate the 2nd and 1st ones...

There are a lot of recalculated numbers here. Let's store and re-use solutions.

Let's store each Fibonacci number *the first time it is calculated*, so that each subsequent time, we can just look it up

```
R = TypeVar('R')

def cache(func: Callable[..., R]) -> Callable[..., R]:
    storage: dict[tuple[Any, ...], R] = {}
    def wrapper(*args: Any) -> R:
        if args in storage:
            return storage[args]
        result = func(*args)
        storage[args] = result
        return result
    return wrapper

@cache
def fibonacci(n: int) -> int:
    if n < 2:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

print(fibonacci(10))
```

Poll: Can we re-use the `@cache` decorator to perform caching for other functions in the same file?

1. Yes
2. Yes, but that other function would share the "cache" of stored values, which are specific to Fibonacci
3. No, it would not run

Easiest way to test this: create a duplicate method called `fibonacci2()`, decorate it, call it after the regular `fibonacci()`, and step through it in the debugger.

Poll:

1. What is your main takeaway from today?

2. What would you like to revisit next time?